# Synthèse de la méthode NCM (Next-Concept Modeling)

Oct 24, 2026 · @Arthur

## Résumé et idée centrale

NCM (Next-Concept Modeling) génère du texte du plus grossier au plus fin, en parallèle à chaque niveau, et annonce 10 à 31× plus de vitesse que les modèles de diffusion de texte (DLM) à qualité comparable. Le papier (arXiv 2610.14162, soumis le 9 octobre 2026, cs.CL) est signé Kumbong, Khare, Mlauzi, Woen, Ding, Buchanan, Arora, Mirhoseini et Ré (Stanford / Hazy Research, avec Together AI).

Le principe se résume en trois points :

- **Autorégressif :** un token à la fois, de gauche à droite.
- **Diffusion (DLM) :** toute la séquence est raffinée à chaque pas, et la qualité chute quand on réduit le nombre de pas.
- **NCM :** le texte est représenté par une hiérarchie de codes discrets. Les niveaux courts portent la structure globale, les niveaux longs ajoutent le détail. Le modèle prédit un niveau à la fois, tous les codes du niveau en parallèle.

La formule « écrire le plan avant les mots » vient de la vidéo source, pas du papier. L'image est juste, mais le « plan » est ici un code latent appris, pas un plan en langage naturel.

## Étape 1 : le tokenizer multi-échelle

Le tokenizer encode le texte en une hiérarchie de codes discrets, par un VQ-VAE résiduel multi-échelle. Les premiers niveaux sont courts et capturent la structure d'ensemble ; les suivants sont plus longs et réduisent l'erreur de reconstruction. Il reprend l'approche de VAR pour les images, avec trois adaptations au texte selon le blog de Hazy Research :

- **Un codebook par niveau :** chaque échelle a son propre vocabulaire de codes, alors que VAR partage un seul codebook entre toutes les échelles.
- **Nested dropout :** il pousse les premiers niveaux à porter de l'information utile sans dépendre des suivants.
- **Reconstruction au niveau du token :** un objectif dédié favorise la récupération exacte du texte.

Le blog montre 11 niveaux dans son exemple de traduction, et la hiérarchie est la même pour tous les exemples.

## Étape 2 : le générateur et l'inférence

Une fois le tokenizer figé, un transformer apprend à prédire chaque niveau à partir de la reconstruction cumulative des niveaux précédents (blog Hazy Research).

- **Entraînement :** tous les niveaux sont traités en une seule passe. Certains codes de conditionnement sont corrompus pour rendre le modèle robuste à ses propres erreurs.
- **Inférence :** le modèle génère un niveau à la fois. Tous les codes d'un niveau sont prédits en parallèle.
- **Décodage final :** le décodeur du tokenizer convertit la hiérarchie complète en texte.

Le nombre de passes est donc égal au nombre de niveaux, pas à la longueur du texte, ce qui explique le gain en nombre d'évaluations de fonction (NFE).

L'animation de traduction du blog illustre le processus sur un exemple allemand → anglais, décodé niveau par niveau. Le BLEU est calculé à chaque niveau et atteint 77,8 au niveau 11 sur 11.

## Résultats annoncés

NCM va 10 à 31× plus vite que les DLM les plus forts, avec une qualité égale ou meilleure sur les tâches testées (papier, blog).

| Tâche | Jeu de données | Résultat annoncé face au meilleur DLM |
|---|---|---|
| Génération libre | LM1B | Meilleure perplexité générative, 21× plus rapide |
| Génération libre | OpenWebText | Perplexité compétitive, 10× plus rapide |
| Résumé | XSum | Même ROUGE-L, 31× plus rapide |
| Traduction | WMT14 De-En | 22,6 BLEU, derrière E2D2 (24,8) et l'autorégressif (25,2) ; 25,2 avec MBR |
| Génération libre | TinyStories, LM1B | Qualité équivalente avec 11× moins d'évaluations de fonction (NFE) |

Autres points du blog :

- **Données d'entraînement :** environ 4,9× moins de tokens sur LM1B et 4× moins sur OpenWebText, tokenizer inclus.
- **Passage à l'échelle :** des résultats préliminaires de 97M, 317M et 725M de paramètres sur LM1B montrent une amélioration régulière de la perplexité générative et de l'entropie unigramme.

- **Décodage MBR :** le décodage par risque bayésien minimal sur plusieurs candidats améliore BLEU et ROUGE. La vitesse de NCM permet d'en générer davantage à budget de temps égal.
- **Mise en garde des auteurs :** la perplexité générative ne capture pas toute la qualité générative.

## Sudoku : le test de planification globale

NCM résout 86,1 % des grilles difficiles, contre 81,3 % pour le meilleur concurrent de diffusion et 1,0 % pour le modèle autorégressif (blog Hazy Research). Le tableau donne le pourcentage de grilles entièrement résolues.

| Difficulté | ARM (autorégressif) | MDLM | DUO | FMLM | NCM |
|---|---|---|---|---|---|
| Facile (40 indices) | 14,6 % | 92,0 % | 96,3 % | 97,7 % | 99,5 % |
| Moyen (35 indices) | 5,1 % | 77,1 % | 84,7 % | 92,8 % | 96,5 % |
| Difficile (30 indices) | 1,0 % | 30,2 % | 58,4 % | 81,3 % | 86,1 % |

NCM est le meilleur à tous les niveaux et se dégrade le moins quand la difficulté augmente. Les auteurs l'attribuent à la planification globale permise par la génération grossier → fin. Le résumé du papier ajoute que NCM y utilise moins d'évaluations de fonction que les baselines.

Lecture prudente : le Sudoku récompense la vision globale des contraintes, ce qui désavantage un modèle gauche-droite par construction. Ce test montre l'intérêt de la planification, pas une supériorité générale sur les LLM.

## Limites et points de vigilance

Les limites les plus nettes sont l'échelle (générateurs d'environ 100M de paramètres, 725M au maximum) et des comparaisons faites surtout face à la diffusion : en traduction, NCM sans décodage MBR reste derrière l'autorégressif et le meilleur modèle de diffusion. Cette section s'appuie sur le texte complet du papier (PDF lu) : l'annexe I, « Limitations & Future Work », et les tableaux du corps du texte.

**Reconnues par les auteurs (annexe I et F.2.3)**

- **Hiérarchie fixe :** la profondeur K et la longueur de chaque niveau sont fixées, donc même une courte continuation traverse toute la hiérarchie. Un calendrier dépendant de l'entrée adapterait le calcul à la complexité.
- **Tokenizer séparé :** il est entraîné à part, ce qui découple l'apprentissage des représentations de l'objectif génératif. Un entraînement de bout en bout est une piste.

- **Échelle :** les expériences principales utilisent des générateurs d'environ 100M de paramètres ; l'étude d'échelle monte à 725M sur LM1B seulement.
- **Artefacts :** les générations sont « généralement fluides » mais montrent des artefacts occasionnels aux transitions entre niveaux grossiers.
- **Corruption des codes :** trop peu ou trop de corruption dégrade la qualité (Figure 13). Une corruption fondée sur les erreurs réelles du modèle est laissée à de futurs travaux.

**Constats tirés des tableaux et des échantillons du papier**

- **Traduction (WMT14 De-En, Table 2) :** NCM obtient 22,6 BLEU avec 11 NFE, contre 25,2 pour l'autorégressif de 99M (64 NFE) et 24,8 pour E2D2, le meilleur modèle non autorégressif. Avec MBR (132 NFE), NCM atteint 25,2, à égalité avec l'autorégressif. Le blog parle de résultats « égaux ou meilleurs » ; le tableau est plus nuancé.
- **Résumé (XSum) :** NCM bat l'autorégressif de 99M en ROUGE-L (28,0 contre 24,4 ; 29,6 avec MBR).
- **Baselines autorégressives petites :** 99M de paramètres en traduction et résumé, 25M au Sudoku, avec des résultats repris d'un autre papier (marqués † dans la Table 2). Les temps mesurés (1 024 séquences, lot de 16, un H100) comparent NCM aux seuls modèles de diffusion ; je n'ai pas trouvé de mesure de temps face à un modèle autorégressif.
- **Tokenizer en goulot d'étranglement :** la reconstruction exacte vaut 98,79 % sur WMT14, 99,79 % sur XSum et 99,86 % sur LM1B (Table 1). Les erreurs sont surtout des substitutions de mots ou d'entités proches, dont l'effet sur le sens dépend du contexte. L'usage des codebooks va de 62 % à 90 %.
- **Qualité à l'échelle des exemples (annexe J) :** une traduction à 18,6 BLEU contient « BMW » à la place de Huawei, « operating operating » et « and and ». Un résumé écrit « James Taylor-Fletcher » au lieu de Gary. Un échantillon non conditionnel sur LM1B (perplexité générative 55,6) est peu cohérent.
- **Métrique :** la perplexité générative est calculée avec GPT-2-large et peut baisser quand la diversité diminue ; les auteurs rapportent donc aussi l'entropie unigramme.
- **Sudoku :** tâche synthétique à vocabulaire de 12 tokens, avec un tokenizer à 100 % de reconstruction ; elle ne dit rien des erreurs du tokenizer sur du texte naturel.

## Contexte : VAR, DiffusionGemma et Mercury 2

NCM transpose au texte l'idée de VAR et se positionne contre la famille des modèles de diffusion de texte, dont DiffusionGemma et Mercury 2 sont des exemples.

| Modèle | Nature | Principe | Chiffres annoncés | Limites affichées |
|---|---|---|---|---|
| NCM | Recherche | Hiérarchie de concepts, niveau par niveau | 10 à 31× plus rapide que les DLM | Implicites : échelle préliminaire |
| VAR | Recherche (image) | Prédiction de l'échelle suivante | FID 18,65 (VQGAN, 227M) → 1,73 (VAR 2B avec rejection sampling ; 1,92 sans) ; environ 20× plus rapide que VQGAN | Tokenizer VQGAN inchangé ; texte et vidéo non traités |
| DiffusionGemma | Modèle ouvert expérimental (Google) | Débruitage de blocs de 256 tokens | Jusqu'à 4× plus rapide, plus de 1 000 tokens/s sur H100 | Explicites : qualité inférieure à Gemma 4, gain réduit en cloud à forte charge |
| Mercury 2 | Produit commercial (Inception) | Raffinement parallèle de tokens | 1 009 tokens/s sur Blackwell, plus de 5× plus rapide | Aucune ; pas de benchmark standard |

NCM est la source la plus détaillée en chiffres mais la plus petite en échelle. DiffusionGemma est la plus transparente sur ses limites. Mercury 2 est la moins vérifiable, ses chiffres venant du fournisseur.

**Nuances sur VAR, d'après le texte complet** (arXiv 2404.02905) :

- Le FID de 18,65 est celui de VQGAN (227M de paramètres, 256 étapes, valeur reprise de MaskGIT), comparé à VAR-d30-re (2,0B, 10 étapes, rejection sampling). Sans rejection sampling, VAR-d30 obtient 1,92.
- Le gain de 20× est mesuré face à VQGAN et ViT-VQGAN (environ 19 s contre 1 s par image). Face à DiT-XL/2 (FID 2,27), le papier annonce un temps 45× supérieur.

- VAR utilise l'architecture VQGAN avec une couche de quantification multi-échelle résiduelle, et une complexité de $O(n^4)$ au lieu de $O(n^6)$ pour la génération autorégressive classique.
- Limites reconnues : le tokenizer est laissé inchangé ; la génération à partir de texte est une piste en cours et la vidéo n'est pas mise en œuvre.

## Références et sources

Pages ouvertes et lues (consultées le 24 octobre 2026) :


1. Kumbong, Khare, Mlauzi, Woen, Ding, Buchanan, Arora, Mirhoseini, Ré. *Next-Concept Modeling: Fast, Parallel Coarse-to-Fine Text Generation*. arXiv:2610.14162 : arxiv.org/abs/2610.14162. Texte complet (PDF) lu le 24 octobre 2026.
2. Hazy Research. *Next-Concept Modeling*, blog, 12 octobre 2026 : hazyresearch.stanford.edu/blog/2026-10-12-ncm.
3. Keyu Tian, Yi Jiang, Zehuan Yuan, Bingyue Peng, Liwei Wang. *Visual Autoregressive Modeling: Scalable Image Generation via Next-Scale Prediction*. arXiv:2404.02905 : arxiv.org/abs/2404.02905. Texte complet (PDF) lu le 24 octobre 2026 ; Peking University et ByteDance.
4. Google. *DiffusionGemma*, blog : blog.google/.../diffusion-gemma-faster-text-generation.
5. Inception. *Introducing Mercury 2*, blog : inceptionlabs.ai/blog/introducing-mercury-2.


Sources secondaires, trouvées par recherche (snippets, non lues en détail) :

- Eden AI, *NCM vs LLM* : edenai.co/post/ncm-vs-llm-next-concept-modeling
- AI Weekly : aiweekly.co/alerts/next-concept-modeling-claims-31-speedup-over-diffusion-lms
- Wiki GitHub des auteurs : github.com/HazyResearch/ncm/wiki

La vidéo d'origine n'a pas pu être consultée (accès bloqué). Titre, auteurs et chiffres de VAR vérifiés dans le PDF (v2, 10 juin 2024) ; le preprint porte la mention « Under review » et aucune conférence.